

Assignment 2 — Encoding & Transformation Impact

Feature Engineering & MLOps · Unit 2, Topics 1 & 2 (Categorical Encoding, Transformations)

Course	Feature Engineering & MLOps
Assignment No.	2
Topic	One-Hot Encoding (2.1) + Transformations & Model Comparison (2.2)
Marks	____ / 100
Due Date	_____

1. Objective

This assignment has two connected parts. In **Part 2.1**, you will one-hot encode the categorical columns of the PrepEdge dataset. In **Part 2.2**, you will apply a numeric transformation to a skewed feature and measure its effect on two different model types — **Linear Regression** and a **Decision Tree Regressor** — trained BEFORE and AFTER the transformation. Your job is to show, with real numbers, that:

- Linear Regression — which fits a straight-line/coefficient-based relationship and is sensitive to feature scale and shape, the same way distance-based algorithms are — **improves** after the skewed feature is transformed.
- The Decision Tree Regressor — which only makes threshold splits — is **largely unaffected** by the same transformation, because a monotonic transform doesn't change which side of a threshold any data point falls on.

This connects directly to Unit 2, Topic 2, Section 10 ("When NOT to transform") — here you will generate the evidence for that claim yourself, on real data.

2. Learning Outcomes

- Correctly one-hot encode nominal categorical features, with train-only fitting discipline.
- Diagnose skewness in a numeric feature and choose an appropriate transformation.
- Design a fair before/after experiment: same train/test split, same random seed, only the feature representation changes.
- Compare model performance using appropriate regression metrics (R^2 and RMSE).
- Explain, in your own words, WHY the two model families respond differently to the same transformation.

3. Dataset

Use `data/raw/student_performance_raw.csv` from your Unit 1 repository — the same PrepEdge dataset you have already been working with. It conveniently contains both categorical columns and a genuinely skewed numeric column in one place:

Column	Role in this assignment
city, course, batch_type	Nominal categorical columns for Part 2.1 (one-hot encoding)
weekly_study_hours	Right-skewed numeric column for Part 2.2 (transformation target)
attendance_pct, prev_exam_score, doubt_sessions_attended	Additional numeric predictors — keep these unchanged in both the before/after models
final_score	Regression target (continuous) for both Linear Regression and Decision Tree

NOTE: `weekly_study_hours` and `prev_exam_score` contain missing values (Unit 1, Session 4). Impute them first (your Assignment 1 `CustomImputer`, or `SimpleImputer`, is fine here) — do this BEFORE the train/test split described in Section 4, fitting only on the training data.

4. Tasks

4.1 — Part 2.1: One-Hot Encoding

1. Load the dataset and identify the nominal categorical columns to encode: `city`, `course`, and `batch_type`. State briefly why these are nominal (no natural order) rather than ordinal.
2. Split the data into train (80%) and test (20%) using a fixed `random_state` so your split is reproducible.
3. Fit `sklearn.preprocessing.OneHotEncoder` (`handle_unknown='ignore'`) on the TRAINING split's categorical columns only, then transform both the train and test splits with the fitted encoder.
4. Report the number of columns in the dataset BEFORE encoding and AFTER encoding, and briefly comment on the increase.
5. Confirm no category from the test set was dropped or caused an error — if your data has any category that appears only in the test split, explain how `handle_unknown='ignore'` handled it.

4.2 — Part 2.2: Transformation & Model Comparison

1. Compute the skewness of `weekly_study_hours`. Confirm it is meaningfully right-skewed (Unit 2, Topic 2, Section 1).
2. Choose and apply ONE transformation to `weekly_study_hours`: `log(np.log1p)`, square root, or Yeo-Johnson (`PowerTransformer`) — state which you chose and why, based on the column's value range (Unit 2, Topic 2, Section 2's decision table).
3. Build TWO versions of your feature set, changing only `weekly_study_hours` and keeping every other column (including your Part 2.1 one-hot columns and the other numeric predictors) identical:
 - **Version A ("Before")** — uses the raw, untransformed `weekly_study_hours`.
 - **Version B ("After")** — uses the transformed `weekly_study_hours` from step 2 above.
4. Using the SAME train/test split from Part 2.1 (same rows, same `random_state`), train FOUR models total:

Model	Feature version	Expected effect of transformation
LinearRegression	Version A (before)	baseline
LinearRegression	Version B (after)	R^2 should improve
DecisionTreeRegressor	Version A (before)	baseline
DecisionTreeRegressor	Version B (after)	R^2 should stay roughly the same

5. For all four models, report both R^2 and RMSE on the TEST split, in one summary table (model × before/after × metric).
6. Create one bar chart (or grouped bar chart) visualizing the R^2 comparison across all four runs, so the pattern is visible at a glance.
7. For the Decision Tree, set a fixed `random_state` and the same `max_depth` in both runs — you are isolating the effect of the transformation, so every other setting must stay identical between the before/after models.

4.3 — Written conclusion (1 short paragraph)

Using your own results table, explain WHY Linear Regression's performance changed after transformation while the Decision Tree's barely moved. Your answer should reference how each model type actually uses a feature's raw values internally (coefficients/gradients vs. threshold splits) — not just restate that "it improved" or "it didn't change."

5. Submission Instructions

1. In your existing course GitHub repository, create a top-level folder named exactly `Assignments` if you have not already done so for Assignment 1.
2. Inside `Assignments`, add your completed notebook as `Assignment2_Encoding_Transformation.ipynb`. Run every cell top-to-bottom with no errors, and keep the outputs (tables, chart) visible before committing.
3. Structure the notebook with clear markdown headers matching Part 2.1, Part 2.2, and your written conclusion, so each part is easy to find and grade.
4. Commit and push your changes:

```
git add Assignments/Assignment2_Encoding_Transformation.ipynb
git commit -m "Assignment 2: Encoding and Transformation impact"
git push
```

Finally, share your GitHub repository link with the instructor by the due date above. Grading is done directly from the `Assignments` folder in your repo — make sure both Assignment 1 and Assignment 2 notebooks are there and pushed before the deadline.

IMPORTANT: Only the notebook committed to your `Assignments` folder on GitHub will be evaluated. Local-only or emailed notebooks will not be graded.

6. Grading Rubric

Criteria	Weight
Part 2.1: correct one-hot encoding, train-only fitting, dimensionality reported	20%
Part 2.2: appropriate transformation chosen and correctly justified	15%
Part 2.2: fair before/after methodology (identical split, seed, model settings)	20%
Part 2.2: correct results table (R^2 & RMSE, all 4 runs) and comparison chart	20%
Written conclusion correctly explains the linear-vs-tree difference	15%
Code quality: clear structure, markdown headers, readable variable names	10%

7. Bonus (Optional, +10% extra credit)

Add a **K-Nearest Neighbors Regressor** (a genuinely distance-based algorithm, unlike Linear Regression which is gradient/coefficient-based) to your before/after comparison. Does KNN show an even larger improvement from the transformation than Linear Regression did? Briefly explain why or why not, referencing how KNN computes distances between rows.

Feature Engineering & MLOps · Assignment 2 · Refer to Unit 2, Topic 1 (Categorical Encoding) and Topic 2 (Transformations) notebooks for the underlying concepts if you get stuck.